SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1 – Case Study Analysis

Course Code:	CSA10 / CSA1004	**Course Title:**	Software Engineering
CO Assessed:	CO1	**Assessment:**	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	**Total Marks:**	25
CO1 Statement:	*Apply advanced methodologies and agile project management techniques to manage software projects effectively*		
Student Name:	Bhuvaneshwari J	**Reg. No.:**	192572141
Date:	10/10/2026	**Duration:**	60 minutes

Code of Conduct

I _____ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

1. **Understand the Problem Statement**
   - Explain the problem in your own words.
   - Identify the objectives of the proposed system.
   - State the scope of the problem.
2. **Identify Key Stakeholders and Challenges**
   - Identify all the stakeholders involved in the system.
   - Explain the roles and responsibilities of each stakeholder.
   - Discuss the major challenges and issues faced by the stakeholders.
3. **Analyze the Current Approach**
   - Describe the existing system or current process.
   - Identify its strengths and limitations.
   - Analyze the problems or gaps that need to be addressed.
4. **Propose a Suitable Solution Using Software Engineering Principles**
   - Design a software-based solution for the given problem.
   - Describe the major functional modules of the proposed system.
   - Identify the functional and non-functional requirements.
   - Explain how software engineering principles such as modularity, scalability, maintainability, reliability, usability, and security are incorporated into your solution.
5. **Justify the Chosen Methodology/Framework**
   - Select an appropriate Software Development Life Cycle (SDLC) model or software development framework (e.g., Agile, Scrum, Waterfall, Spiral, DevOps).

- Justify why the selected methodology/framework is suitable for the proposed system.
- Explain how it supports successful project development and maintenance.

6. **Discuss Expected Outcomes and Limitations**
   - Describe the expected benefits of the proposed solution.
   - Explain how the solution addresses the identified challenges.
   - Discuss potential risks, implementation challenges, and limitations.
   - Suggest possible future enhancements.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/frame work and provides	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing;

	strong technical justification.			report lacks organization and clarity.

Criteria	**Mark**
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25

Smart Construction Project Monitoring System – Case Study Analysis
Course: CSA10 / CSA1004 – Software Engineering | **CO Assessed:** CO1 | **Assessment:** Tool 1 – Case Study Analysis
Problem Statement No. 29: Smart Construction Project Monitoring System

1. Understand the Problem Statement

1.1 The Problem in My Own Words

A construction project involves many moving parts at once: several contractors, many workers, large quantities of materials, a fixed budget, a strict schedule and a set of milestones. In most firms this information is scattered across site diaries, paper registers, spreadsheets and phone messages, so it reaches the project manager late, incomplete or inconsistent. A cement shortage, an absent crew or a slipping milestone is noticed only after it has caused a delay, and delays turn into cost overruns and missed deadlines.

The task is to design a **Smart Construction Project Monitoring System**: one centralised platform that tracks activities, workers, materials, expenses, milestones and risks, and gives project managers real-time progress information so they can act early.

1.2 Objectives of the Proposed System

- Manage construction projects, phases and tasks in one place; track milestones (planned vs. actual).
- Monitor material availability, usage and reorder levels; record workforce information and attendance.
- Track project expenses against budget; identify delays automatically; manage risks and issues.
- Generate progress, cost, material and workforce reports.

1.3 Scope

In scope: project, milestone and task tracking; material inventory and suppliers; worker and contractor records; budget and expense tracking; delay detection and alerts; risk and issue register; role-based dashboards and PDF/Excel reports; web interface for managers and a mobile-friendly interface for site engineers. **Out of scope (first release):** payroll, full accounting and tax, CAD/BIM design, IoT/drone integration and automatic payments.

2. Identify Key Stakeholders and Challenges

2.1 Stakeholders and Their Roles

Stakeholder	Role and Responsibilities
Project Manager	Plans the project, monitors milestones, budget and risks, takes corrective decisions; primary dashboard user.
Site Engineer / Supervisor	Updates daily progress, attendance, material usage and site issues from the field.
Contractors / Workers	Execute assigned work packages; their attendance and productivity are recorded.

Stakeholder	Role and Responsibilities
Store Manager / Suppliers	Maintain stock, receive and issue materials, raise reorder requests, deliver on time.
Finance / Accounts Team	Approves budgets, verifies expenses, tracks cost variance and payments.
Client / Senior Management	Fund and review progress; need transparency and portfolio-level reports.
Safety & Quality Officer	Records risks, incidents and inspection results; ensures regulatory compliance.
System Administrator	Manages users, roles, backups and maintenance.

2.2 Major Challenges

- **Project managers:** no real-time visibility, decisions based on outdated data, difficulty coordinating contractors.
- **Site and store staff:** duplicate manual entry, poor connectivity, inaccurate stock leading to shortages or wastage.
- **Finance:** expenses recorded late, so overruns are discovered only at the end.
- **Clients and management:** low transparency; delays, risks and rework are found late and information is not shared.

3. Analyze the Current Approach

3.1 Existing System

Site supervisors write progress in logbooks and attendance registers; the store keeps ledgers or Excel sheets; accounts maintain separate expense spreadsheets; schedules are kept in Excel or MS Project and updated occasionally. Project managers collect information through calls, emails and weekly or monthly meetings and compile reports by hand.

3.2 Strengths and Limitations

Strengths	Limitations
Simple, familiar, needs little training	Data is delayed, fragmented and in different formats
Low initial cost, no software investment	High risk of errors, duplication, loss or manipulation
Works without internet or electricity	No real-time tracking or automatic delay detection
Experienced engineers have good ground knowledge	Schedule, material, labour and cost data are not linked
Flexible for small projects	Slow manual reports; reactive risk handling; weak access control; poor scalability

3.3 Gaps to Be Addressed

The key gaps are: no single source of truth; no live visibility; late delay detection; material shortages and wastage; uncontrolled costs; unmanaged risks; slow reporting; and weak security with no audit trail. The proposed system must close each of these with centralised data, live dashboards, planned-vs-actual alerts, stock and budget monitoring, a risk register, automated reports and role-based access.

4. Propose a Suitable Solution Using Software Engineering Principles

4.1 Solution Overview

A web-based, role-driven system on a **three-tier architecture**: presentation layer (responsive web/mobile UI), application layer (business logic exposed through REST APIs) and data layer (relational database). Data entered by site engineers, store managers and finance staff flows into one database; a monitoring engine compares planned and actual values and pushes alerts and dashboards to managers. Suggested stack: React, Spring Boot/Node.js, MySQL/PostgreSQL, JWT authentication.

4.2 Major Functional Modules

Module	Description
1. User & Access Management	Login, role-based permissions, audit trail.
2. Project Management	Create projects and phases, assign managers, set dates and budget.
3. Milestone & Task Tracking	Define tasks, update % completion, compare planned vs. actual dates.
4. Material Management	Catalogue, stock in/out, balance, suppliers, low-stock alerts.
5. Workforce Management	Worker/contractor records, skills, attendance, task assignment.
6. Expense & Budget Management	Record expenses by category, variance and overrun alerts.
7. Delay Detection & Alerts	Rule-based engine flags overdue tasks; email/SMS/in-app notifications.
8. Risk & Issue Management	Risk register with probability, impact, owner, mitigation and status.
9. Dashboard & Reporting	Live KPIs and charts; progress, cost, material and workforce reports.

4.3 Functional Requirements

- **FR1:** Create and manage projects, phases, milestones and tasks. **FR2:** Record actual progress and compare with the plan.
- **FR3:** Maintain material stock and alert below reorder level. **FR4:** Record worker details, attendance and assignments.
- **FR5:** Record expenses and compare with budget. **FR6:** Automatically identify delays and notify the project manager.
- **FR7:** Log, assign, track and close risks and issues. **FR8:** Generate PDF/Excel reports. **FR9:** Provide secure login and role-based dashboards.

4.4 Non-Functional Requirements

Category	Requirement
Performance	Dashboard loads within 3 seconds; updates visible within seconds of entry.
Scalability	Supports many projects and concurrent users; horizontally scalable services.
Reliability	99.5% uptime, daily backups, recovery from failure.
Security	HTTPS, encrypted passwords, role-based access, audit logs.
Usability / Portability	Simple responsive UI, minimal training, works on modern browsers and devices.
Maintainability	Modular, documented code with automated tests.

4.5 Software Engineering Principles Applied

Principle	How It Is Applied
Modularity	Independent modules (project, material, workforce, expense, risk, reporting) with clear interfaces; each can be built, tested and changed separately.
Scalability	Layered architecture, stateless REST APIs, indexed database, cloud deployment with load balancing.
Maintainability	Low coupling, high cohesion, coding standards, Git version control, documentation and unit tests.
Reliability	Input validation, exception handling, transactions, regular backups and logging.
Usability	Role-specific dashboards, simple forms, colour-coded status, mobile-friendly design.
Security	Authentication, role-based authorisation, encrypted communication and credentials, audit trails.

5. Justify the Chosen Methodology / Framework

5.1 Selected Model: Agile – Scrum

I select the **Agile methodology using the Scrum framework**, with two-week sprints, a prioritised product backlog, sprint planning, daily stand-ups, sprint reviews and retrospectives. Project managers act as the Product Owner, the development team delivers working increments, and a Scrum Master facilitates the process.

5.2 Justification

- **Changing and evolving requirements:** project managers, engineers and finance staff will refine what they need once they see working screens; Scrum welcomes change, whereas Waterfall locks requirements early.
- **Many stakeholders:** regular sprint reviews let all stakeholders give feedback frequently, ensuring the system matches real site practice.
- **Incremental value:** the most important modules (milestone tracking, material monitoring, delay alerts) can be released first, and expenses, risk management and advanced reports follow in later sprints.
- **Risk reduction:** early and continuous delivery exposes usability and integration problems early, instead of at the end.
- **Team collaboration and transparency:** daily stand-ups and a visible backlog keep the team aligned, and the system itself is developed in the same spirit of visibility that it provides to construction teams.

5.3 How It Supports Development and Maintenance

- Each sprint ends with a tested, potentially shippable increment, which gives a continuous feedback loop.
- Continuous integration and automated testing keep quality high as the code grows.
- The product backlog naturally absorbs change requests, bug fixes and new features after release, which makes maintenance a continuing sprint activity rather than a separate phase.
- Retrospectives drive continuous improvement of both the product and the process.

Sprint roadmap (example): Sprint 1 – login, roles, project setup; Sprint 2 – milestones and tasks; Sprint 3 – materials and workforce; Sprint 4 – expenses and budget; Sprint 5 – delay alerts and risk management; Sprint 6 – dashboards, reports, testing and deployment.
Alternatives considered: Waterfall is too rigid for changing requirements; Spiral is heavy for the project size; a hybrid with DevOps (CI/CD) is used alongside Scrum for automated builds and deployment.

5.1 Selected Model: Agile – Scrum

I select **Agile with the Scrum framework**: two-week sprints, a prioritised product backlog, sprint planning, daily stand-ups, sprint reviews and retrospectives. The project manager acts as Product Owner, the development team delivers working increments and a Scrum Master facilitates.

5.2 Justification

- **Evolving requirements:** managers, engineers and finance staff will refine needs after seeing working screens; Scrum welcomes change, whereas Waterfall freezes requirements early.
- **Many stakeholders:** sprint reviews give frequent feedback so the system matches real site practice.
- **Incremental value:** milestone tracking, material monitoring and delay alerts are released first; expenses, risks and advanced reports follow.
- **Risk reduction:** early, continuous delivery exposes usability and integration problems sooner.

5.3 Support for Development and Maintenance

Each sprint ends with a tested, potentially shippable increment, supported by continuous integration and automated testing. The backlog absorbs bug fixes and change requests after release, so maintenance continues as normal sprint work, and retrospectives improve both product and process. **Example roadmap:** Sprint 1 – login, roles, project setup; 2 – milestones and tasks; 3 – materials and workforce; 4 – expenses and budget; 5 – delay alerts and risks; 6 – dashboards, reports, testing and deployment. Waterfall was rejected as too rigid and Spiral as too heavy for this project size; DevOps (CI/CD) is used alongside Scrum.

6. Discuss Expected Outcomes and Limitations

6.1 Expected Benefits and How Challenges Are Addressed

Challenge	How the Solution Addresses It / Outcome
Late or scattered information	Central database with real-time updates → improved project visibility
Late detection of delays	Delay engine and notifications → early identification of delays
Material shortages and wastage	Stock tracking and reorder alerts → better resource utilisation
Budget overruns	Expense-vs-budget tracking and variance reports → improved cost control
Unmanaged risks; poor coordination	Risk register, shared dashboards, role-based access → better coordination
Slow reporting	Automated reports → faster decisions and higher probability of timely completion

6.2 Risks, Implementation Challenges and Limitations

- User resistance and training needs; poor site connectivity (mitigated by offline entry with later sync).
- Output depends on accurate, timely data entry; initial development, hosting and training costs.
- Security and privacy of financial and project data; scope creep (mitigated by backlog prioritisation).
- First version relies on manual data entry and excludes IoT and automatic payments.

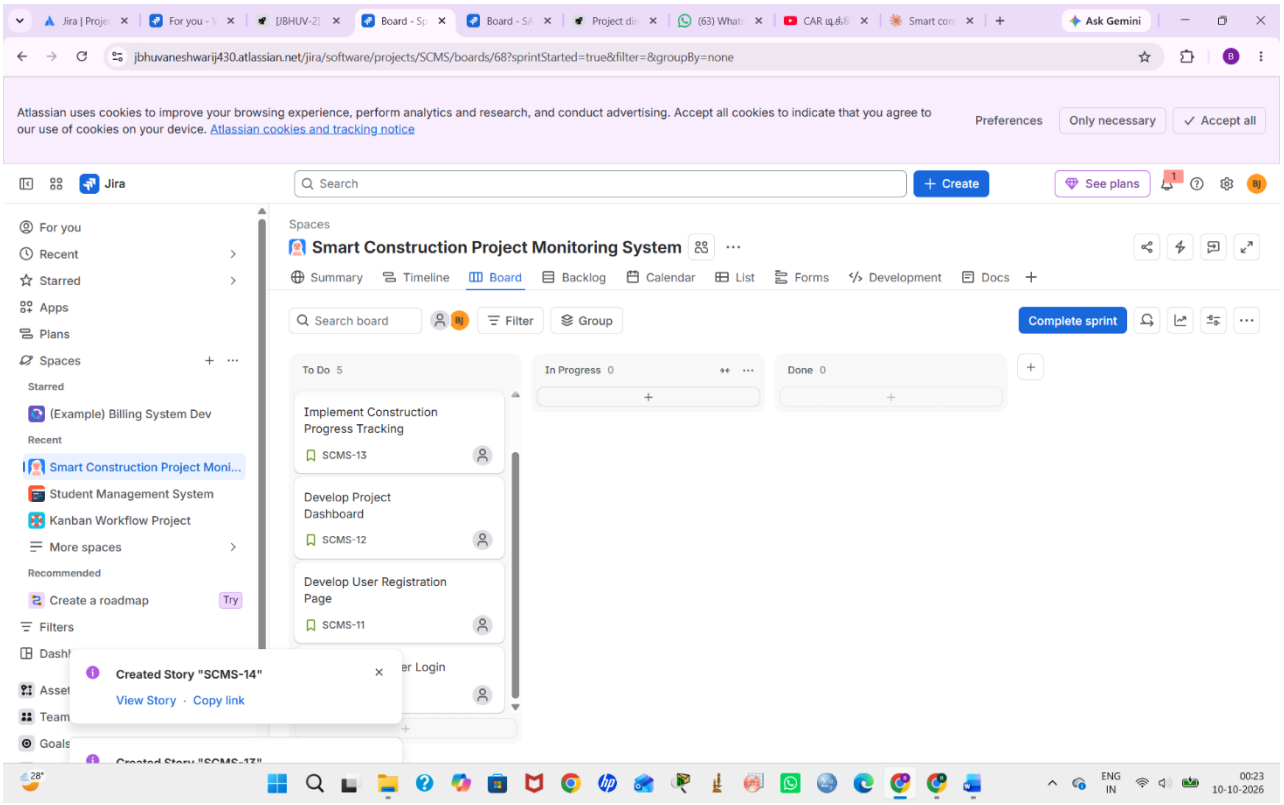
6.3 Future Enhancements

IoT sensors, RFID and GPS for automatic material and equipment tracking; drone and image-based progress monitoring; AI/ML models to predict delays, cost overruns and risks; native mobile app with full offline mode; integration with ERP, BIM and payroll; digital invoice and approval workflows; multi-language and voice-based updates.

Conclusion

The Smart Construction Project Monitoring System replaces fragmented manual tracking with a single, modular and secure platform that gives project managers live insight into milestones, materials, workforce, expenses and risks. Built with Agile Scrum, it delivers value early, adapts to feedback and improves cost control, coordination and the likelihood of on-time completion.

Sprint started



Sprint progress

